Universidade Católica de Brasília

Ciência da Computação

Arthur Freitas Araújo
Arthur Martins Fonseca Dias
Caio Seabra de Queiroz
Marina Souza Ramos

Funcionamento da memória cache e técnicas de otimização

Relatório técnico-acadêmico apresentado à Universidade Católica de Brasília, como parte dos requisitos avaliativos da unidade curricular de Arquitetura de Computadores.
Professor: Weldes Lima Oliveira.

Brasília – DF
2026

Sumário

1 Introdução 3
1.1 Contextualização 3
1.2 Importância para o desempenho 3
1.3 Objetivos 4
1.3.1 Objetivo geral 4
1.3.2 Objetivos específicos 4
1.4 Organização do trabalho 4
2 Fundamentação teórica 5
2.1 Hierarquia de memória 5
2.2 Princípio da localidade 5
2.2.1 Localidade temporal 5
2.2.2 Localidade espacial 6
2.3 Mapeamento 7
2.3.1 Mapeamento direto 7
2.3.2 Mapeamento totalmente associativo 8
2.3.3 Mapeamento associativo por conjunto 9
2.4 Políticas de escrita 9
2.4.1 Write-through 9
2.4.2 Write-back 10
2.5 Taxa de acerto e penalidade de falha 10
2.6 Desafios comuns 11
2.6.1 Cache thrashing 11
2.6.2 Coerência de cache em múltiplos núcleos 12
2.6.3 Falso compartilhamento 13
2.7 Técnicas de otimização 14
3 Mapeamento e diagrama arquitetural 16
3.1 Hierarquia de memória 16
3.2 Divisão do endereço 16
3.3 Esquema de cada mapeamento 17
3.4 Fluxo de acesso à cache 18
3.5 Cache da máquina usada na PoC 18
4 Metodologia e desenvolvimento 20
4.1 Ambiente de teste 20
4.2 Ferramentas de medição 20
4.3 Algoritmos implementados 21
4.3.1 Percurso linha por linha 21

4.3.2 Percurso coluna por coluna 23
4.4 Passo a passo da PoC 24
4.5 Métricas coletadas 25
4.6 Limitações do método 25
5 Discussão dos resultados 26
5.1 Análise da comparação 26
5.2 Do código em C ao hardware 26
5.3 Limitações do método 27
5.4 Limitações físicas 27
5.5 Impacto no desenvolvimento de software de alto desempenho 27
6 Conclusão 28
6.1 Síntese do aprendizado 28
6.2 Lições aprendidas 28
6.3 Implicações práticas 28

Referências 30

1 Introdução

1.1 Contextualização

A memória cache é uma memória volátil de alta velocidade, porém com pequena capacidade de armazenamento. O tempo de acesso a um dado nela contido é muito menor que se o mesmo estivesse na memória principal. O propósito do uso da memória cache é minimizar a disparidade existente entre a velocidade com que o processador executa instruções e a velocidade com que dados são lidos e gravados na memória principal. Apesar de ser uma memória de acesso rápido, a capacidade de armazenamento das memórias cache é limitada em função do seu alto custo.

A memória cache armazena uma pequena parte do conteúdo da memória principal. Toda vez que o processador faz referência a um dado armazenado na memória, é verificado, primeiramente, se o mesmo encontra-se na memória cache. Caso o processador encontre o dado (*cache hit*), não há necessidade do acesso à memória principal, diminuindo assim o tempo de acesso.

Se a informação desejada não estiver presente na cache, o acesso à memória principal é obrigatório (*cache miss*). Neste caso, o processador, a partir do dado referenciado, transfere um bloco de dados da memória principal para a cache.

A localidade é a tendência de o processador, ao longo da execução de um programa, referenciar instruções e dados na memória principal localizados em endereços próximos. Esta tendência é justificada pela alta incidência de estruturas de repetição, chamadas a sub-rotinas e acesso a estruturas de dados como vetores e tabelas (MACHADO; MAIA, 2013).

1.2 Importância para o desempenho

Apesar de existir neste mecanismo um tempo adicional para a transferência de dados entre as diferentes memórias, este tempo é compensado pela melhora do desempenho, justificado pelo alto percentual de referências a endereços que são resolvidos na cache. O princípio da localidade garante que após a transferência de um novo bloco da memória principal para a cache haverá uma alta probabilidade de *cache hits* em futuras referências, otimizando, assim, o tempo de acesso ao dado.

A maioria dos processadores apresenta uma arquitetura de memória cache com múltiplos níveis. O funcionamento desta arquitetura tem como base o princípio de que quanto menor é a capacidade de armazenamento da memória cache, mais rápido é o acesso ao dado; contudo, a probabilidade da ocorrência de *cache hits* é menor (MACHADO; MAIA, 2013).

1.3 Objetivos

1.3.1 Objetivo geral

Analisar e comparar o desempenho de dois métodos de acesso aos elementos de uma matriz bidimensional: o percurso linha por linha e o percurso coluna por coluna, demonstrando como a organização dos dados na memória e a forma como eles são acessados influenciam o desempenho de um programa.

1.3.2 Objetivos específicos

- Apresentar os conceitos fundamentais da memória cache, incluindo hierarquia de memória, princípio da localidade e políticas de escrita.
- Implementar algoritmos em linguagem C para comparar diferentes padrões de acesso à memória.
- Medir o tempo de execução e a taxa de falhas de cache (*cache misses*) de cada algoritmo.
- Analisar a influência da hierarquia de memória, especialmente da memória cache, no desempenho computacional.

1.4 Organização do trabalho

O Capítulo 2 apresenta a fundamentação teórica sobre a hierarquia de memória, princípio da localidade, mapeamento e políticas de escrita. O Capítulo 3 traz os diagramas arquiteturais. O Capítulo 4 descreve a metodologia e a Prova de Conceito (PoC). O Capítulo 5 discute os resultados obtidos e suas limitações. O Capítulo 6 reúne as conclusões do grupo.

2 Fundamentação teórica

2.1 Hierarquia de memória

A hierarquização da cache em múltiplos níveis é uma solução para aumentar o desempenho no funcionamento das memórias caches. O nível de cache mais alto é chamado de L1 (*Level 1*), com baixa capacidade de armazenamento e com altíssima velocidade de acesso. O segundo nível, L2 (*Level 2*), possui maior capacidade de armazenamento, porém com velocidade de acesso inferior a L1. Quando a UCP necessita acessar um dado na memória principal, primeiramente é verificado se o dado encontra-se na cache L1. Caso o dado seja encontrado, obtém-se um excelente desempenho no acesso à informação, porém, caso o dado não seja encontrado, a busca prossegue para a L2.

Um processador pode ser projetado com diversos níveis de cache, conforme especificação do fabricante. Como exemplos, o processador da Motorola Power PC G4 possui cache de nível 2, enquanto o processador Intel Xeon MP possui cache de nível 3 (MACHADO; MAIA, 2013).

2.2 Princípio da localidade

O *caching* talvez seja o exemplo mais importante da grande ideia da predição. Ele conta com o princípio da localidade para tentar encontrar os dados desejados nos níveis mais altos da hierarquia de memória, e oferece mecanismos para garantir que, quando a predição for errada, ela encontra e usa os dados apropriados dos níveis mais baixos da hierarquia da memória. As taxas de acerto da predição de cache nos computadores modernos, normalmente, são mais altas do que 95% (PATTERSON; HENNESSY, 2017).

2.2.1 Localidade temporal

Na oitava referência, temos demandas em conflito para um bloco. A palavra no endereço 18 (10010_{bin}) deve ser trazida para o bloco de cache 2 (010_{bin}). Logo, ela precisa substituir a palavra no endereço 26 (11010_{bin}), que já está no bloco de cache 2 (010_{bin}). Esse comportamento permite que uma cache tire proveito da localidade temporal: palavras recentemente acessadas substituem palavras menos referenciadas recentemente (PATTERSON; HENNESSY, 2017).

Índice	V	Tag	Dados
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

a. O estado inicial do cache depois de power-on

Índice	V	Tag	Dados
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	**10_{bin}**	**Memória (10110_{bin})**
111	N		

b. Depois de lidar com uma falta de endereço (10110_{bin})

Índice	V	Tag	Dados
000	N		
001	N		
010	Y	**11_{bin}**	**Memória (11010_{bin})**
011	N		
100	N		
101	N		
110	Y	10_{bin}	Memória (10110_{bin})
111	N		

c. Depois de lidar com uma falta de endereço (11010_{bin})

Índice	V	Tag	Dados
000	Y	**10_{bin}**	**Memória (10000_{bin})**
001	N		
010	Y	11_{bin}	Memória (11010_{bin})
011	N		
100	N		
101	N		
110	Y	10_{bin}	Memória (10110_{bin})
111	N		

d. Depois de lidar com uma falta de endereço (10000_{bin})

Índice	V	Tag	Dados
000	Y	10_{bin}	Memória (10000_{bin})
001	N		
010	Y	11_{bin}	Memória (11010_{bin})
011	Y	**00_{bin}**	**Memória (00011_{bin})**
100	N		
101	N		
110	Y	10_{bin}	Memória (10110_{bin})
111	N		

e. Depois de lidar com uma falta de endereço (00011_{bin})

Índice	V	Tag	Dados
000	Y	10_{bin}	Memória (10000_{bin})
001	N		
010	Y	**10_{bin}**	**Memória (10010_{bin})**
011	Y	00_{bin}	Memória (00011_{bin})
100	N		
101	N		
110	Y	10_{bin}	Memória (10110_{bin})
111	N		

f. Depois de lidar com uma falta de endereço (10010_{bin})

Figura 1 – Exemplo de localidade temporal (estados de uma cache)

Fonte: Elaborado pelos autores (2026).

2.2.2 Localidade espacial

Blocos maiores exploram a localidade espacial para diminuir as taxas de falhas. Aumentar o tamanho de bloco normalmente diminui a taxa de falhas. A taxa de falhas pode subir posteriormente se o tamanho de bloco se tornar uma fração significativa do tamanho de cache, uma vez que o número de blocos que pode ser armazenado na cache se tornará pequeno e haverá uma grande competição entre esses blocos. Explicando de outra forma: a localidade espacial entre as palavras em um bloco diminui com um bloco muito grande; por conseguinte, os benefícios na taxa de falhas se tornam menores (PATTERSON; HENNESSY, 2017).

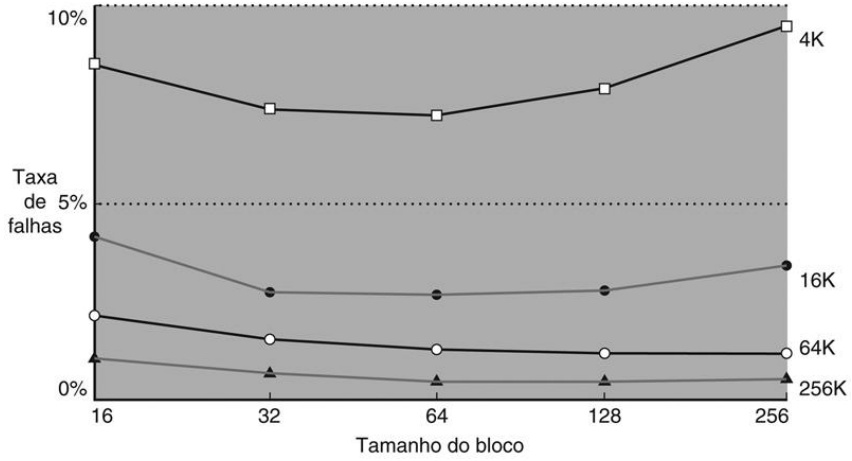


Figura 2 – Impacto do tamanho do bloco na taxa de falhas

Fonte: Elaborado pelos autores (2026).

2.3 Mapeamento

2.3.1 Mapeamento direto

A maneira mais simples de atribuir um local na cache para cada palavra da memória é atribuir um local na cache baseado no endereço da palavra na memória. Essa estrutura de cache é chamada de mapeamento direto, já que cada local da memória é mapeado diretamente para um local exato na cache.

(Endereço de bloco) módulo (Número de blocos na cache)

Se o número de entradas na cache for uma potência de dois, então, o módulo pode ser calculado simplesmente usando os $\log_2$ bits menos significativos do endereço (PATTERSON; HENNESSY, 2017).

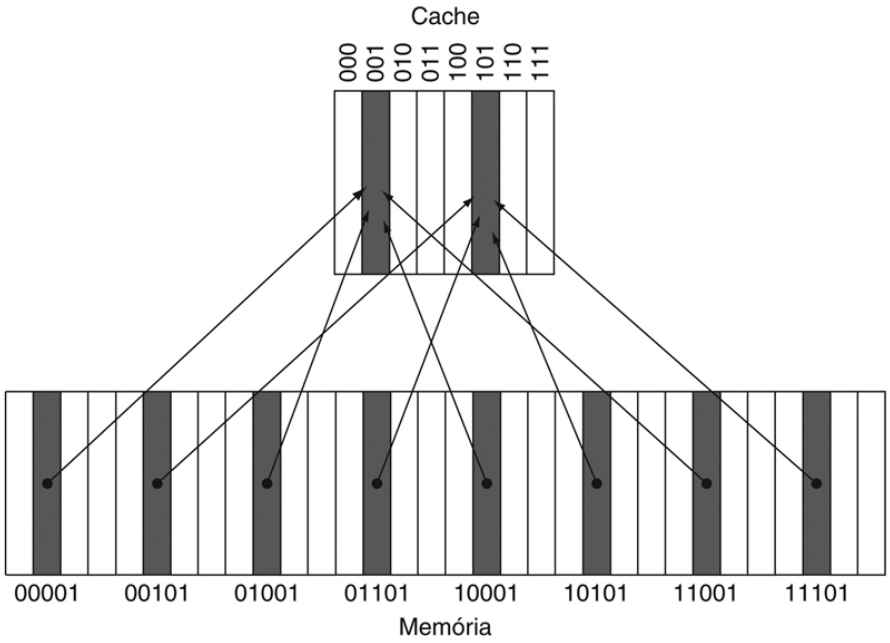


Figura 3 – Exemplo prático de mapeamento direto na memória

Fonte: Elaborado pelos autores (2026).

Como há oito palavras na cache, um endereço X é mapeado para a palavra de cache X, módulo 8. Ou seja, os $\log_2(8) = 3$ bits menos significativos são usados como o índice da cache. Assim, os endereços 00001_{bin}, 01001_{bin}, 10001_{bin} e 11001_{bin} são todos mapeados para a entrada 001_{bin} da cache, enquanto os endereços 00101_{bin}, 01101_{bin}, 10101_{bin} e 11101_{bin} são todos mapeados para a entrada 101_{bin} da cache.

2.3.2 Mapeamento totalmente associativo

Até agora, quando colocamos um bloco na cache, usamos um esquema de posicionamento simples: um bloco só pode entrar exatamente em um local na cache. Como já dissemos, esse esquema é chamado de mapeamento direto porque qualquer endereço de bloco na memória é diretamente mapeado para um único local, no nível superior da hierarquia. Existe, na verdade, toda uma faixa de esquemas para posicionamento de blocos. Em um extremo está o mapeamento direto, em que um bloco só pode ser posicionado exatamente em um local.

No outro extremo está um esquema em que um bloco pode ser posicionado em qualquer local na cache. Para encontrar um determinado bloco em uma cache totalmente associativa, todas as entradas da cache precisam ser pesquisadas. Para tornar a pesquisa possível, ela é feita em paralelo com um comparador associado a cada entrada da cache. Esses comparadores aumentam muito o custo do hardware (PATTERSON; HENNESSY, 2017).

2.3.3 Mapeamento associativo por conjunto

A faixa intermediária de projetos entre a cache diretamente mapeada e a cache totalmente associativa é chamada de associativa por conjunto. Em uma cache associativa por conjunto, existe um número fixo de locais (pelo menos dois) onde cada bloco pode ser colocado. Um bloco é diretamente mapeado para um conjunto e, então, uma correspondência é pesquisada em todos os blocos no conjunto (PATTERSON; HENNESSY, 2017).



Figura 4 – Comparação lado a lado: mapeamento direto, associativo e totalmente associativo

Fonte: Elaborado pelos autores (2026).

2.4 Políticas de escrita

2.4.1 Write-through

A maneira mais simples de manter consistentes a memória principal e a cache é sempre escrever os dados na memória e na cache. Essas escritas levarão muito tempo, talvez mais de 100 ciclos de *clock*. Uma solução é usar um *buffer* de escrita (HENNESSY; PATTERSON, 2017).

Com um esquema de *write-through*, toda escrita faz com que os dados sejam escritos na memória principal. Essas escritas levarão muito tempo, talvez mais de 100 ciclos de *clock* de processador, e tornariam o processador consideravelmente mais lento.

Uma solução para esse problema é usar um *buffer* de escrita (ou *write buffer*), que armazena os dados enquanto estão esperando para serem escritos na memória. Após escrever os dados na cache e no *buffer* de dados, o processador pode continuar a execução. Quando uma escrita na memória principal é concluída, a entrada no *buffer* de escrita é liberada.

Se o *buffer* de escrita estiver cheio quando o processador atingir uma escrita, o processador precisará sofrer *stall* até que haja uma posição vazia no *buffer* de escrita. Para reduzir a ocorrência desses *stalls*, os processadores normalmente aumentam a profundidade do *buffer* de escrita para além de uma única entrada.

Considere uma falha em uma cache *write-through*. A estratégia mais comum é alocar um bloco na cache, chamado alocar na escrita. O bloco é apanhado da memória e depois a parte apropriada do bloco é sobrescrita. Uma estratégia alternativa é atualizar a parte do bloco na memória, mas não colocá-la em cache, o que se chama não alocar na escrita. A motivação é que, às vezes, os programas escrevem blocos de dados, como quando o sistema operacional zera uma página de memória. Nesses casos, a busca associada com a falha de escrita inicial pode ser desnecessária.

2.4.2 Write-back

A alternativa é o esquema *write-back*, no qual, quando ocorre uma escrita, o novo valor é escrito apenas no bloco da cache. O bloco modificado é escrito no nível inferior da hierarquia quando ele é substituído. Pode melhorar o desempenho, mas é mais complexo de implementar (HENNESSY; PATTERSON, 2017).

Os esquemas *write-back* podem melhorar o desempenho, especialmente quando os processadores podem gerar escritas tão rápido ou mais rápido do que as escritas podem ser tratadas pela memória principal; entretanto, um esquema *write-back* é mais complexo de implementar do que um esquema *write-through*.

Implementar *stores* de modo realmente eficaz em uma cache que usa uma estratégia *write-back* é mais complexo do que em uma cache *write-through*. Em uma cache *write-back*, precisamos escrever o bloco novamente na memória se os dados na cache estiverem modificados e tivermos uma falha de cache.

Em uma cache *write-back*, como não podemos substituir o bloco, os *stores* ou exigem dois ciclos (um ciclo para verificar um acerto seguido de um ciclo para efetivamente realizar a escrita) ou exigem um *buffer* de escrita para conter esses dados na prática, permitindo que a *store* leve apenas um ciclo por meio de um *pipeline* de memória.

Muitas caches *write-back* também incluem *buffers* de escrita usados para reduzir a penalidade de falha quando uma falha substitui um bloco modificado. Em casos como esse, o bloco modificado é movido para um *buffer write-back* associado com a cache enquanto o bloco requisitado é lido da memória. Depois, o *buffer write-back* é escrito novamente na memória.

2.5 Taxa de acerto e penalidade de falha

Essa técnica é baseada na observação de que o processador normalmente precisa de apenas uma palavra do bloco de cada vez. Essa estratégia é a da impaciência: não espere até que o bloco inteiro seja carregado para então enviar a palavra solicitada e reiniciar o processador. Aqui estão duas estratégias específicas:

- **Palavra crítica primeiro:** Solicite primeiro a palavra que falta e envie-a para o processador assim que ela chegar; deixe o processador continuar a execução enquanto

preenche o restante das palavras no bloco.

- **Reinício antecipado:** Busque as palavras na ordem normal, mas, assim que a palavra solicitada chegar, envie-a para o processador e deixe que ele continue a execução.

Geralmente, essas técnicas só beneficiam projetos com grandes blocos de cache, pois o benefício é baixo, a menos que os blocos sejam grandes. Observe que, normalmente, as caches continuam a satisfazer os acessos a outros blocos enquanto o restante do bloco está sendo preenchido.

Entretanto, dada a proximidade espacial, existe boa chance de que a próxima referência seja para o restante do bloco. Assim como as caches sem bloqueio, a penalidade de falta não é simples de calcular. Quando existe uma segunda solicitação da palavra crítica primeiro, a penalidade de falta efetiva é o tempo não sobreposto da referência até que a segunda parte chegue. Os benefícios da palavra crítica primeiro e de reinício antecipado dependem do tamanho do bloco e da probabilidade de outro acesso à parte do bloco que ainda não foi acessada.

2.6 Desafios comuns

2.6.1 Cache thrashing

A função da memória cache é agilizar a execução de programas, armazenando cópias de blocos de dados da memória principal em uma memória cache mais rápida, embora de menor capacidade. O gráfico na Figura 5 ilustra a taxa de falhas (*miss ratio*) de uma memória cache em função do seu tamanho e associatividade. Quanto menor for o valor da taxa de falhas, melhor será o desempenho da cache. As falhas encontradas em uma memória cache podem ser classificadas em três categorias: falhas compulsórias, por capacidade ou por conflito (também chamadas de 3 Cs) (HENNESSY; PATTERSON, 2017).

- **Falhas compulsórias:** São falhas que ocorrem sempre no primeiro acesso a um bloco, já que o bloco nunca foi armazenado na memória cache.

- **Falhas devido à capacidade:** São falhas que ocorrem quando a cache não consegue armazenar todos os blocos necessários à execução de um programa, por não ter capacidade suficiente.

- **Falhas por conflitos ou colisão:** São falhas que ocorrem quando diversos blocos competem pela mesma posição (conjunto) na memória cache. Esse tipo de falha não ocorre em caches totalmente associativas, pois neste caso um bloco pode ser armazenado em qualquer linha da memória cache.

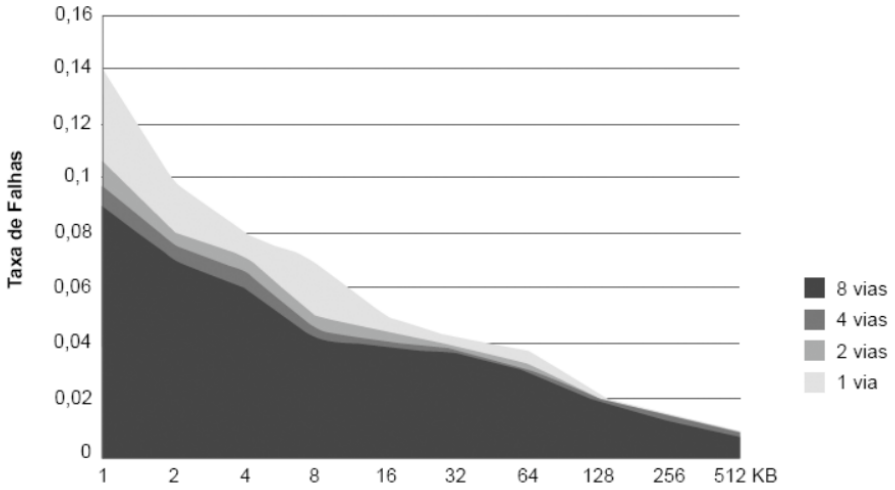


Figura 5 – Impacto da associatividade na taxa de falhas

Fonte: Elaborado pelos autores (2026).

2.6.2 Coerência de cache em múltiplos núcleos

Nos processadores modernos, com mais de um núcleo, é comum a existência de um terceiro nível de cache (L3), compartilhada por todos os núcleos. O tempo de acesso com acerto a essa cache é da ordem de algumas dezenas de ciclos de relógio. O uso de caches multiníveis é um recurso comum nos modernos processadores, mas que pode resultar na diminuição da taxa de acerto nos níveis mais próximos ao processador, pelo uso de caches com capacidade menor nesses níveis para acompanhar as velocidades crescentes de relógio dos processadores. A arquitetura e o desempenho final de uma memória cache devem ser adequadamente avaliados por meio de simulações, antes da sua implementação física no processador (SILVA; BORGES, 2024).

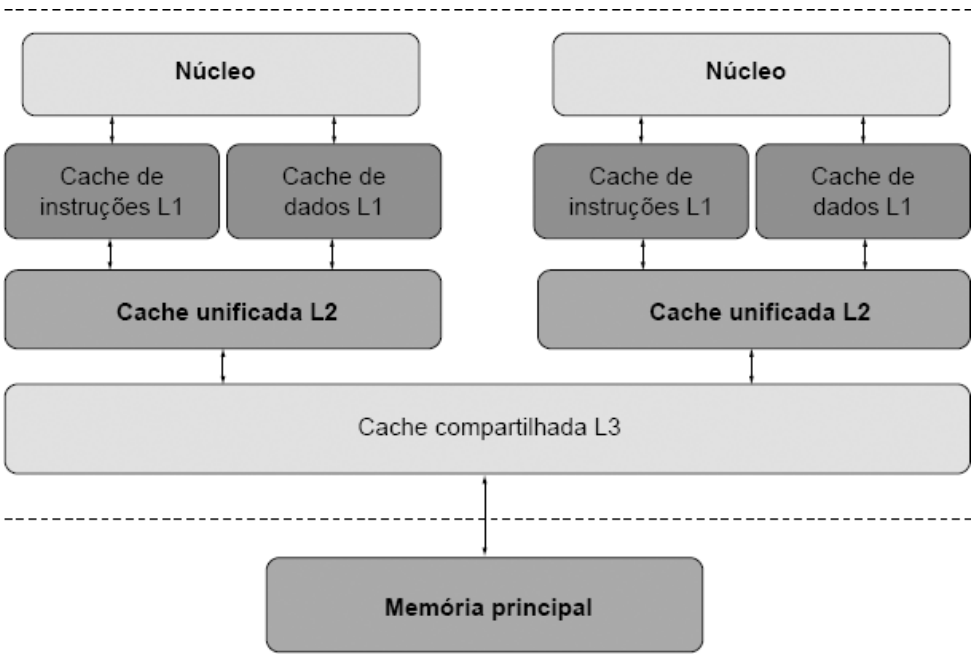


Figura 6 – Caches multiníveis

Fonte: Elaborado pelos autores (2026).

2.6.3 Falso compartilhamento

No início dos tempos a memória cache está vazia, ou seja, não possui nenhuma informação armazenada e todas as suas linhas estão marcadas como inválidas. As instruções e dados vão sendo então gradativamente copiados dos níveis inferiores, como a memória principal e secundária, para a memória cache, à medida que são utilizados pelo processador. Na realidade, o processador inicia a busca das instruções ou dos dados na suposição de que estejam armazenados na memória cache. Se forem encontrados na memória cache (houve um acerto), a informação desejada é transferida para o processador. Mas se os dados ou a instrução não estiverem na memória cache (houve uma falha), então o processador aguarda, enquanto a instrução/dados desejados são copiados para a memória cache e depois transferidos para o processador.

Durante a busca da informação que está faltando na cache, em vez de trazer somente a instrução ou dados solicitados pelo processador, um bloco inteiro é copiado da memória principal e armazenado em uma linha da cache, que então é marcada como válida. O objetivo é minimizar a taxa de falhas nos próximos acessos, seguindo o princípio da localidade espacial. O tamanho de um bloco é um parâmetro de projeto nas memórias caches, tamanhos usuais são 32, 64 e 128 bytes. Note que um bloco trazido da memória principal recebe o nome de linha quando armazenado na memória cache, sendo, portanto, de mesmo tamanho (SILVA; BORGES, 2024).

2.7 Técnicas de otimização

Essas otimizações aumentam a largura de banda da cache seja pelo *pipelining* do acesso à cache ou pela ampliação da cache com múltiplos bancos, permitindo múltiplos acessos por ciclo de *clock*; essas otimizações estão associadas às abordagens *superpipelined* e superescalar para aumentar o *throughput* de instrução. Elas visam principalmente a cache L1, onde a largura de banda de acesso restringe o *throughput* de instrução.

O *pipelining* L1 permite um ciclo de *clock* mais alto, ao custo de uma maior latência. O *pipelining* da cache de instruções efetivamente aumenta o número de estágios do *pipeline*, levando a uma penalidade maior nos desvios mal previstos. De modo correspondente, o *pipelining* da cache de dados leva a mais ciclos de *clock* entre a emissão do *load* e o uso dos dados.

Para lidar com múltiplos acessos à cache de dados por *clock*, podemos dividir a cache em bancos independentes, cada um suportando um acesso independente. Os bancos foram originalmente usados para melhorar o desempenho da memória principal e agora são usados dentro de chips DRAM modernos, bem como com caches. O Intel Core i7 contém quatro bancos na cache L1 (para admitir até 2 acessos de memória por *clock*) (HENNESSY; PATTERSON, 2019).

Uma cache sem bloqueio ou cache sem travamento aumenta os benefícios em potencial de tal esquema, permitindo que a cache de dados continue a fornecer acertos de cache durante uma falta. Essa otimização de "acerto sob falta" reduz a penalidade de falta efetiva, sendo útil durante uma falta, em vez de ignorar as solicitações do processador. Uma opção sutil e complexa é que a cache pode reduzir ainda mais a penalidade de falta efetiva se puder sobrepor múltiplas faltas: uma otimização "acerto sob múltiplas faltas" ou "falta sob falta". A segunda opção só será benéfica se o sistema de memória puder atender a múltiplas faltas; a maior parte dos processadores de alto desempenho (como os processadores Intel Core) geralmente suporta ambos, enquanto muitos dos processadores de baixo nível oferecem somente suporte limitado sem bloqueio na cache L2 (HENNESSY; PATTERSON, 2019).

Até aqui, nossas técnicas têm exigido a mudança do hardware. Essa próxima técnica reduz as taxas de falta sem quaisquer mudanças no hardware. Essa redução mágica vem do software otimizado a solução favorita do projetista de hardware! A diferença de desempenho cada vez maior entre os processadores e a memória principal tem inspirado os projetistas de compiladores a investigar a hierarquia de memória para ver se as otimizações em tempo de compilação podem melhorar o desempenho. Mais uma vez, a pesquisa está dividida entre as melhorias nas faltas de instrução e as melhorias nas faltas de dados. As otimizações apresentadas a seguir são encontradas em muitos compiladores modernos.

Em vez de operar sobre linhas ou colunas inteiras de um *array*, os algoritmos bloqueados operam sobre submatrizes ou blocos. O objetivo é maximizar os acessos aos dados

carregados na cache antes que eles sejam substituídos (HENNESSY; PATTERSON, 2019).

As caches sem bloqueio reduzem efetivamente a penalidade de falta, sobrepondo a execução com o acesso à memória. Outra técnica é fazer a pré-busca (*prefetch*) dos itens antes que o processador os solicite. Tanto as instruções quanto os dados podem ter sua busca antecipada, seja diretamente nas caches, seja diretamente em um buffer externo, que pode ser acessado mais rapidamente do que a memória principal.

Frequentemente, a pré-busca de instrução é feita no hardware fora da cache. Em geral, o processador apanha dois blocos em uma falta: o bloco solicitado e o próximo bloco consecutivo. O bloco solicitado é colocado na cache de instruções, e o bloco cuja busca foi antecipada é colocado no buffer do fluxo de instruções. Se o bloco solicitado estiver presente no buffer do fluxo de instruções, a solicitação de cache original será cancelada, o bloco será lido do buffer de fluxo e a próxima solicitação de pré-busca será emitida (HENNESSY; PATTERSON, 2019).

3 Mapeamento e diagrama arquitetural

3.1 Hierarquia de memória

O sistema de memória de dados modelado com base no Intel i7 consiste em uma cache L1 de 32 KiB, com latência de acesso de quatro ciclos. A cache L2 (compartilhada com instruções) é de 256 KiB, com uma latência de acesso de 10 ciclos de *clock*. A cache L3 tem 2 MiB e uma latência de acesso de 36 ciclos. Todas as caches são associativas por conjunto de oito vias e têm tamanho de bloco de 64 bytes. Permitir um acerto sob falta reduz a penalidade de falta em 9% para os *benchmarks* inteiros e de 12,5% para os de ponto flutuante. Permitir um segundo acerto melhora esses resultados para 10% e 16%, e permitir 64 resulta em pouca melhoria adicional (HENNESSY; PATTERSON, 2019).

3.2 Divisão do endereço

As caches possuem uma *tag* de endereço em cada *frame* de bloco, que indica o endereço do bloco. A *tag* de cada bloco da cache que poderia conter a informação desejada é verificada, para saber se corresponde ao endereço de bloco do processador. Em geral, todas as *tags* possíveis são pesquisadas em paralelo, pois a velocidade é crítica.

É preciso haver um meio de saber que um bloco de cache não tem informação válida. O procedimento mais comum é acrescentar um bit de validade à *tag* para dizer se essa entrada contém ou não um endereço válido. Se o bit não for definido, não poderá haver uma combinação nesse endereço.

Antes de passar à próxima pergunta, vamos explorar o relacionamento de um endereço do processador com a cache. A primeira divisão ocorre entre o endereço de bloco e o *offset* de bloco. O endereço do *frame* de bloco pode ser dividido ainda mais, em campo de *tag* e campo de índice. O campo de *offset* de bloco seleciona os dados desejados do bloco, o campo de índice seleciona o conjunto e o campo de *tag* é comparado com ele para ver se há um acerto. Embora a comparação pudesse ser feita sobre mais de um endereço do que a *tag*, isso não é necessário, devido ao seguinte:

- O *offset* não deve ser usado na comparação, pois ou o bloco inteiro está presente ou não, e por isso todos os *offsets* de bloco resultam em uma combinação por definição.
- A verificação do índice é redundante, pois foi usada para selecionar o conjunto a ser verificado. Um endereço armazenado no conjunto 0, por exemplo, precisa ter 0 no campo de índice ou não poderia ser armazenado no conjunto 0; o conjunto 1 precisa ter um valor de índice igual a 1, e assim por diante. Essa otimização economiza hardware e energia, reduzindo o tamanho da memória para a *tag* de cache.

Endereço de bloco		Offset de bloco
Tag	Índice	

Tabela 1 – As três partes de um endereço em uma cache associativa em conjunto ou com mapeamento direto.

Fonte: Elaborado pelos autores (2026).

A *tag* é usada para verificar todos os blocos no conjunto, e o índice é usado para selecionar o conjunto. O *offset* de bloco é o endereço dos dados desejados dentro do bloco. As caches totalmente associativas não possuem campo de índice.

Se o tamanho total da cache for mantido igual, o aumento da associatividade aumentará o número de blocos por conjunto, diminuindo assim o tamanho do índice e aumentando o tamanho da *tag*. Ou seja, o limite de índice de *tag* se move para a direita aumentando-se a associatividade, com a extremidade das caches totalmente associativas não possuindo campo de índice (HENNESSY; PATTERSON, 2019).

3.3 Esquema de cada mapeamento

Podemos pensar em cada estratégia de posicionamento de bloco como uma variação da associatividade por conjunto. Uma cache diretamente mapeada é simplesmente uma cache associativa por conjunto de uma via: cada entrada de cache contém um bloco, e cada conjunto possui um elemento. Uma cache totalmente associativa com m entradas é simplesmente uma cache associativa por conjunto de m vias; ele tem um conjunto com m blocos, e uma entrada pode residir em qualquer bloco dentro desse conjunto. O tamanho total da cache em blocos é igual ao número de conjuntos multiplicado pela associatividade. Portanto, para uma cache de tamanho fixo, aumentar a associatividade diminui o número de conjuntos enquanto aumenta o número de elementos por conjunto (HENNESSY; PATTERSON, 2017).

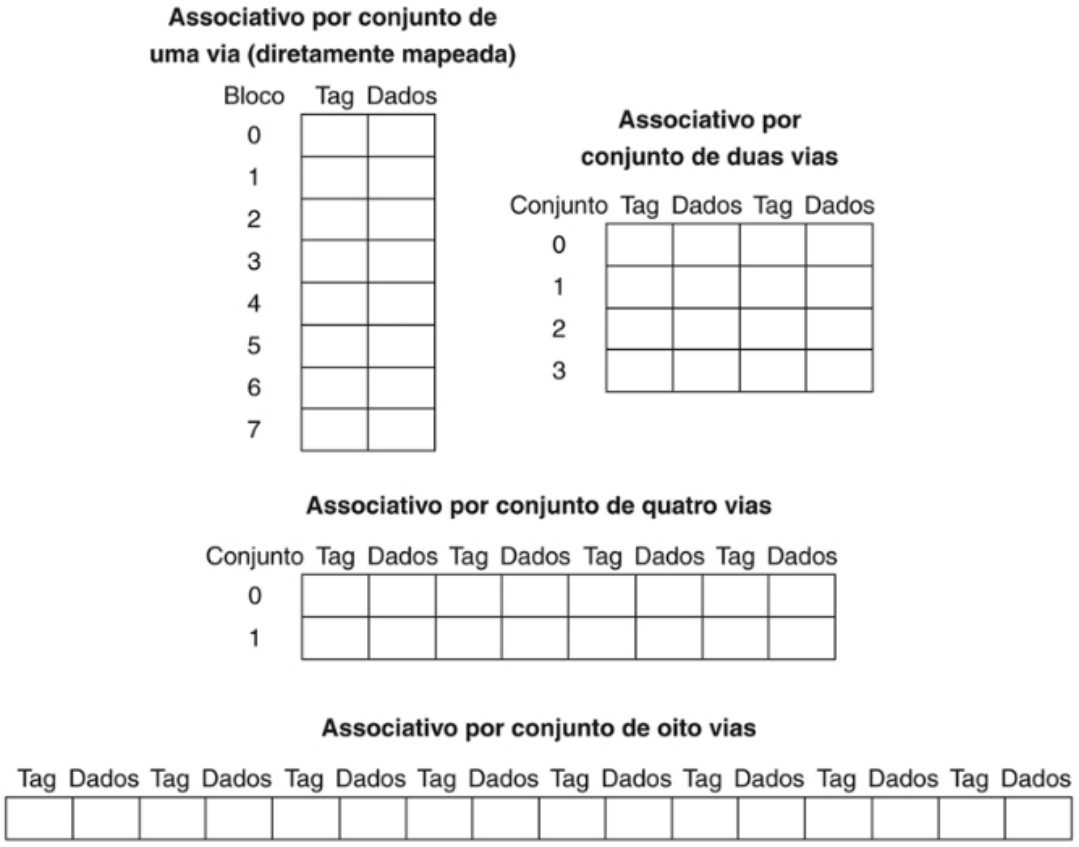


Figura 7 – Esquemas de mapeamento de cache

Fonte: Elaborado pelos autores (2026).

3.4 Fluxo de acesso à cache

A *tag* da cache é comparada com a parte superior do endereço para determinar se a entrada na cache corresponde ao endereço requisitado. Como a cache tem 2^{10} (ou 1024) palavras e um tamanho de bloco de 1 palavra, 10 bits são usados para indexar a cache, deixando $32 - 10 - 2 = 20$ bits para serem comparados com a *tag*. Se a *tag* e os 20 bits superiores do endereço forem iguais e o bit de validade estiver ligado, então, a requisição é um acerto na cache e a palavra é fornecida para o processador. Caso contrário, ocorre uma falha.

A unidade de controle precisa detectar uma falha de cache e processá-la buscando os dados requisitados da memória (ou, como veremos, de uma cache de nível inferior). Se a cache reportar um acerto, o computador continua usando os dados como se nada tivesse acontecido (PATTERSON; HENNESSY, 2017).

3.5 Cache da máquina usada na PoC

A máquina utilizada para os testes da PoC possui um processador Intel Core i5-12500H (12ª geração) e sistema operacional Fedora Linux. De acordo com o comando `lscpu`, a

hierarquia de cache é composta por 448 KiB de cache L1 de dados (L1d), 640 KiB de cache L1 de instruções (L1i), 9 MiB de cache L2 e 18 MiB de cache L3 compartilhada.

4 Metodologia e desenvolvimento

A metodologia utilizada neste trabalho consiste no desenvolvimento de uma Prova de Conceito (PoC) com o objetivo de analisar e comparar o desempenho de dois métodos de acesso aos elementos de uma matriz bidimensional: o percurso linha por linha e o percurso coluna por coluna. A proposta busca demonstrar como a organização dos dados na memória e a forma como eles são acessados podem influenciar diretamente o desempenho de um programa.

Para realizar essa comparação, foram implementados algoritmos em linguagem C, utilizando uma matriz com dimensões previamente definidas. Os dois métodos executam operações sobre os mesmos elementos, porém seguindo ordens diferentes de acesso à memória. Dessa forma, é possível observar diferenças no tempo de execução e compreender a influência da hierarquia de memória, especialmente da memória cache, no desempenho computacional.

4.1 Ambiente de teste

Os testes foram realizados em um ambiente computacional com o objetivo de garantir condições semelhantes para a execução dos dois algoritmos. Para isso, foram utilizados o mesmo computador, sistema operacional, compilador e configurações de execução. A linguagem C foi escolhida por permitir maior controle sobre o acesso à memória.

4.2 Ferramentas de medição

Para avaliar o desempenho dos algoritmos, foram utilizadas ferramentas de medição de tempo de execução disponíveis na linguagem C e no ambiente de desenvolvimento. A medição do tempo permite identificar quanto tempo cada algoritmo leva para percorrer todos os elementos da matriz. Essa informação é importante para compreender como diferentes estratégias de acesso aos dados podem produzir resultados distintos, mesmo quando realizam a mesma quantidade de operações. Além disso, podem ser utilizadas ferramentas de monitoramento de desempenho do sistema para observar aspectos como utilização do processador e comportamento da memória cache. Essas informações complementam os resultados obtidos pela medição do tempo de execução. É importante realizar várias execuções de cada algoritmo e calcular uma média dos resultados, reduzindo a influência de possíveis variações ocasionais do sistema.

4.3 Algoritmos implementados

Foram desenvolvidos dois algoritmos com a finalidade de comparar diferentes formas de percorrer uma matriz bidimensional. Na linguagem C, as matrizes são armazenadas na memória seguindo uma organização denominada *row-major order*.

4.3.1 Percurso linha por linha

O primeiro algoritmo realiza o acesso aos elementos da matriz percorrendo inicialmente todas as colunas de uma linha antes de avançar para a próxima linha. Como os elementos de uma mesma linha estão armazenados em posições consecutivas da memória, o processador consegue aproveitar melhor os dados carregados previamente para a memória cache.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 2048 // Tamanho da matriz (2048x2048)

int main()
{
    // Alocacao dinamica para evitar estouro de pilha
    int **matriz = (int **)malloc(N * sizeof(int *));
    for (int i = 0; i < N; i++)
    {
        matriz[i] = (int *)malloc(N * sizeof(int));
    }

    // Inicializacao
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            matriz[i][j] = 1;
        }
    }

    // Medicao de tempo
    clock_t inicio = clock();
    long long soma = 0;

    // PERCURSO LINHA POR LINHA (Aproveita a localidade espacial)
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            soma += matriz[i][j];
        }
    }

    clock_t fim = clock();
    double tempo = (double)(fim - inicio) / CLOCKS_PER_SEC;

    printf("Soma (Linha por Linha): %lld\n", soma);
    printf("Tempo de execucao: %f segundos\n", tempo);

    // liberar memoria
    for (int i = 0; i < N; i++)
        free(matriz[i]);
    free(matriz);
```

```
48
49      return 0;
50  }
```

Código 4.1 – Código em C para o percurso linha por linha

4.3.2 Percurso coluna por coluna

O segundo algoritmo percorre uma coluna inteira antes de avançar para a próxima. Embora todos os elementos sejam acessados, essa estratégia não acompanha a ordem em que os dados estão organizados na memória, apresentando menor aproveitamento da memória cache.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 2048 // Tamanho da matriz (2048x2048)

int main() {
    // Alocacao dinamica
    int **matriz = (int **)malloc(N * sizeof(int *));
    for (int i = 0; i < N; i++) {
        matriz[i] = (int *)malloc(N * sizeof(int));
    }

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            matriz[i][j] = 1;
        }
    }

    clock_t inicio = clock();
    long long soma = 0;

    // PERCURSO COLUNA POR COLUNA (Desfavorece a localidade espacial)
    for (int j = 0; j < N; j++) {
        for (int i = 0; i < N; i++) {
            soma += matriz[i][j];
        }
    }

    clock_t fim = clock();
    double tempo = (double)(fim - inicio) / CLOCKS_PER_SEC;

    printf("Soma (Coluna por Coluna): %lld\n", soma);
    printf("Tempo de execucao: %f segundos\n", tempo);

    for (int i = 0; i < N; i++) free(matriz[i]);
    free(matriz);

    return 0;
}
```

Código 4.2 – Código em C para o percurso coluna por coluna

4.4 Passo a passo da PoC

1. Criação e inicialização da matriz bidimensional.

2. Implementação do percurso linha por linha (dois laços de repetição).
3. Desenvolvimento do percurso coluna por coluna (invertendo a ordem dos laços).
4. Medição do tempo de execução de cada algoritmo.
5. Análise dos dados considerando a organização da memória.

4.5 Métricas coletadas

As principais métricas consideradas foram o tempo de execução e a diferença de desempenho entre os dois métodos. Também foram considerados indicadores como a quantidade de acertos (*cache hits*) e falhas (*cache misses*).

4.6 Limitações do método

A metodologia apresenta limitações relacionadas ao hardware utilizado, fatores como processos em segundo plano, temperatura do processador e otimizações do compilador. O tamanho da matriz também influencia, pois se os dados couberem completamente na cache, a diferença entre os algoritmos será reduzida.

5 Discussão dos resultados

A discussão dos resultados tem como objetivo interpretar os dados obtidos durante a execução dos algoritmos e relacioná-los aos conceitos de arquitetura de computadores e organização da memória. A comparação entre os percursos linha por linha e coluna por coluna permite observar que a ordem de acesso aos dados influencia diretamente o desempenho de aplicações que trabalham com matrizes.

Tabela 2 – Comparação entre os dois padrões de acesso à memória

Métrica	**Linha por linha**	**Coluna por coluna**
Tempo de execução (s)	0,006974	0,010819
Soma calculada	4194304	4194304
L1 cache misses	61.204	5.444.797
LLC cache misses	126.655	320.348

Fonte: Elaborado pelos autores com dados do `perf stat` (2026).

5.1 Análise da comparação

Os resultados demonstram uma diferença significativa de desempenho entre os dois métodos. O percurso linha por linha executou a tarefa em aproximadamente 0,006 segundos, enquanto o percurso coluna por coluna levou cerca de 0,011 segundos. Isso representa um tempo cerca de 55% maior para o algoritmo que percorre as colunas.

5.2 Do código em C ao hardware

A execução de um programa em C envolve uma interação direta entre o software e os componentes físicos. Quando um programa acessa um elemento de uma matriz, o processador precisa obter o dado na hierarquia de memória. Na linguagem C, a organização *row-major* faz com que o percurso linha por linha aproveite melhor os dados já carregados na cache.

No percurso linha por linha, o processador acessa posições consecutivas da memória, favorecendo o aproveitamento dos blocos de dados. No percurso coluna por coluna, os acessos ocorrem em posições mais distantes, provocando mais falhas de cache. Dessa forma, mesmo com a mesma complexidade assintótica ($O(n \times m)$), os desempenhos práticos são diferentes.

5.3 Limitações do método

Os resultados apresentados possuem limitações, como a dependência das configurações do compilador e as características do processador. Variações nos tempos de execução podem ocorrer devido ao sistema operacional e à execução de outros processos.

5.4 Limitações físicas

As limitações físicas estão relacionadas às características dos componentes de hardware. A memória cache possui capacidade limitada, enquanto a memória RAM apresenta maior latência. A largura de banda da memória e o tamanho dos blocos transferidos também influenciam o desempenho.

5.5 Impacto no desenvolvimento de software de alto desempenho

O estudo demonstra que a organização dos dados e a escolha dos padrões de acesso à memória são fundamentais. A utilização adequada da memória cache pode reduzir o tempo de execução e melhorar a eficiência dos recursos computacionais, especialmente em aplicações científicas, processamento de imagens e inteligência artificial.

6 Conclusão

O desenvolvimento deste trabalho permitiu analisar a influência dos padrões de acesso à memória no desempenho de programas desenvolvidos em linguagem C. A partir da implementação dos algoritmos de percurso linha por linha e coluna por coluna, foi possível estudar como a organização dos dados na memória interfere na execução de operações sobre matrizes bidimensionais. Os conceitos estudados demonstram que pequenas alterações na estrutura de um programa podem produzir diferenças significativas de desempenho, especialmente quando são utilizados grandes conjuntos de dados.

6.1 Síntese do aprendizado

Durante o desenvolvimento da atividade, foi possível compreender a importância da hierarquia de memória e sua influência na execução de programas. O estudo demonstrou que a memória cache desempenha um papel fundamental na redução do tempo de acesso aos dados, permitindo que o processador trabalhe de maneira mais eficiente.

Também foi possível compreender que a linguagem C utiliza uma organização de memória na qual os elementos de uma mesma linha de uma matriz são armazenados de forma sequencial. Dessa maneira, o percurso linha por linha tende a proporcionar melhor aproveitamento da memória cache em comparação ao percurso coluna por coluna.

6.2 Lições aprendidas

Uma das principais lições aprendidas foi que a eficiência de um algoritmo não depende apenas de sua complexidade teórica. Embora os dois métodos implementados apresentem a mesma complexidade assintótica, o comportamento durante a execução pode ser diferente devido à organização dos dados na memória.

Outro aprendizado importante foi a necessidade de realizar testes práticos para avaliar o desempenho de um programa. A análise teórica permite formular hipóteses, mas as medições são fundamentais para verificar como o código se comporta em um ambiente real.

6.3 Implicações práticas

Os conhecimentos adquiridos durante o trabalho podem ser aplicados em diferentes áreas da computação, principalmente em programas que realizam operações com grandes volumes de dados. Em aplicações de processamento de imagens, cálculos científicos, inteligência artificial e simulações computacionais, a organização dos dados e a forma de acesso à memória podem influenciar diretamente o tempo necessário para concluir

uma tarefa. Assim, utilizar padrões de acesso que favoreçam a localidade espacial pode contribuir para melhorar o desempenho das aplicações.

Conclui-se que o desenvolvimento de software eficiente exige não apenas conhecimentos de programação, mas também compreensão sobre o funcionamento do hardware. A análise realizada evidencia a importância de considerar a interação entre código, processador e memória durante o desenvolvimento de soluções computacionais.

Referências

HENNESSY, J. L.; PATTERSON, D. A. *Organização e Projeto de Computadores: a interface hardware/software*. 5. ed. Rio de Janeiro: GEN LTC, 2017.

HENNESSY, J. L.; PATTERSON, D. A. *Arquitetura de Computadores: Uma Abordagem Quantitativa*. 6. ed. Rio de Janeiro: GEN LTC, 2019.

Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. [S.l.], 2024. Disponível em: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html. Acesso em: 8 nov. 2026.

MACHADO, F. B.; MAIA, L. P. *Arquitetura de Sistemas Operacionais*. 5. ed. Rio de Janeiro: LTC, 2013.

MONTEIRO, M. A. *Introdução à Organização de Computadores*. 5. ed. Rio de Janeiro: LTC, 2007.

PATTERSON, D. A.; HENNESSY, J. L. *Organização e Projeto de Computadores: a interface hardware/software*. 5. ed. Rio de Janeiro: Elsevier, 2017.

SILVA, G. P. d.; BORGES, J. A. d. S. *Arquitetura e Organização de Computadores: Uma Introdução*. 1. ed. Rio de Janeiro: LTC, 2024.

STALLINGS, W. *Arquitetura e Organização de Computadores*. 10. ed. São Paulo: Pearson Education do Brasil, 2017.

TANENBAUM, A. S.; AUSTIN, T. *Organização Estruturada de Computadores*. 6. ed. São Paulo: Pearson Universidades, 2013.

Valgrind Developers. *Cachegrind: a cache and branch-prediction profiler*. [S.l.], 2024. Manual de referência. Disponível em: https://valgrind.org/docs/manual/cg-manual.html. Acesso em: 8 nov. 2026.

WEBER, R. F. *Fundamentos de Arquitetura de Computadores*. 4. ed. Porto Alegre: Bookman, 2012.